

preprocess

February 11, 2026

```
[7]: import pandas as pd
      from itertools import islice
      from pyexpat import features
      import torch
      import numpy as np
      from torchvision import transforms
      import pandas as pd
      import cv2 as cv
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns
      from joblib import Parallel, delayed
      from scipy.stats import skew
      from sklearn.preprocessing import MinMaxScaler
```

```
[8]: train_df = pd.read_csv('./dataset/train.csv')
      det_df = pd.read_csv('./dataset/train_det.csv')
      gt_df = pd.read_csv('./dataset/train_gt.csv')
```

```
[9]: test_df = pd.read_csv('./dataset/test.csv')
      test_det_df = pd.read_csv('./dataset/test_det.csv')
```

```
[12]: train_df.head()
```

```
[12]:   Unnamed: 0  video_id  video_name  image_width  image_height  frame_id  \
0           0         0  ADL-Rundle-6         1920          1080         0
1           1         0  ADL-Rundle-6         1920          1080         1
2           2         0  ADL-Rundle-6         1920          1080         2
3           3         0  ADL-Rundle-6         1920          1080         3
4           4         0  ADL-Rundle-6         1920          1080         4
```

```
      image_path
0  ./dataset/train/ADL-Rundle-6/img1/000001.jpg
1  ./dataset/train/ADL-Rundle-6/img1/000002.jpg
2  ./dataset/train/ADL-Rundle-6/img1/000003.jpg
3  ./dataset/train/ADL-Rundle-6/img1/000004.jpg
4  ./dataset/train/ADL-Rundle-6/img1/000005.jpg
```

```
[13]: det_df.head()
```

```
[13]: Unnamed: 0  frame_id  object_id  x_coordinate  y_coordinate  width  \
0          0          0          -1          1689          385  146.620
1          1          0          -1          1303          503   61.514
2          2          0          -1          1258          569   40.123
3          3          0          -1           31          525  113.370
4          4          0          -1          1800          483   94.660

      height  confidence  x_init  y_init  z_init  video_id
0  332.710      67.567   -1.0   -1.0   -1.0          0
1  139.590      29.439   -1.0   -1.0   -1.0          0
2   91.049      19.601   -1.0   -1.0   -1.0          0
3  257.270      17.013   -1.0   -1.0   -1.0          0
4  214.810      11.949   -1.0   -1.0   -1.0          0
```

```
[12]: import torchvision.transforms.functional as F
import random

class DataAugmentation:
    def __init__(self, is_train=True):
        self.is_train = is_train

    def __call__(self, image, target):
        if not self.is_train:
            return image, target

        # 1. Color Jittering (Only affects image)
        if random.random() > 0.5:
            brightness = random.uniform(0.8, 1.2)
            contrast = random.uniform(0.8, 1.2)
            image = F.adjust_brightness(image, brightness)
            image = F.adjust_contrast(image, contrast)

        # 2. Horizontal Flip (Affects image and boxes)
        if random.random() > 0.5:
            image = F.hflip(image)
            width = image.shape[-1]
            boxes = target["boxes"]
            # Flip: x_min_new = width - x_max_old; x_max_new = width - x_min_old
            boxes[:, [0, 2]] = width - boxes[:, [2, 0]]
            target["boxes"] = boxes

        # 3. Random Crop (Simplified as "Random Resized Crop" style)
        # Note: True random cropping requires complex box filtering.
        # For object detection, it's safer to use ColorJitter and Flips first.
```

```
return image, target
```

```
[13]: import torch
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torch.utils.data import DataLoader, Dataset
import pandas as pd
import numpy as np
import cv2
import os
from PIL import Image

# --- Configuration ---
DEVICE = torch.device('cuda') if torch.cuda.is_available() else torch.
    ↪device('cpu')
NUM_EPOCHS = 1
BATCH_SIZE = 4 # Start with 4 for 1080p images on 16GB VRAM; increase to 8 if
    ↪stable
LEARNING_RATE = 0.005
NUM_CLASSES = 2 # 1 class (object) + 1 background. Change if you have specific
    ↪classes.
# If your dataset distinguishes between cars/pedestrians, increase this.

# --- 1. Custom Dataset Class ---
class CustomMOTDataset(Dataset):
    def __init__(self, root_dir, index_csv, gt_csv, transforms=None):
        """
        Args:
            root_dir (string): Directory with all the images.
            index_csv (string): Path to train.csv (frames list).
            gt_csv (string): Path to train_gt.csv (annotations).
            transforms (callable, optional): Optional transform to be applied
    ↪on a sample.
        """
        self.root_dir = root_dir
        self.transforms = transforms

        # Load DataFrames
        self.frames_df = pd.read_csv(index_csv)
        self.gt_df = pd.read_csv(gt_csv)

        # Ensure we can map frames to annotations quickly
        # Assuming both CSVs share 'video_id' and 'frame_id' or similar keys
        # We create a dictionary for faster lookup during training
        self.image_data = self.frames_df.reset_index(drop=True)

    def __getitem__(self, idx):
```

```

# 1. Get Image Info
row = self.image_data.iloc[idx]
video_id = row['video_id']
frame_id = row['frame_id']

img_path = row['image_path']

if not os.path.exists(img_path):
    raise FileNotFoundError(f"Image not found: {img_path}")

image = cv2.imread(img_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB).astype(np.float32)
image /= 255.0 # Normalize to [0, 1]

# 2. Get Annotations for this frame
# Filter GT dataframe for this specific video and frame
# Adjust column names 'video_id'/'frame_id' to match your CSV headers
→ exactly
boxes_data = self.gt_df[
    (self.gt_df['video_id'] == video_id) &
    (self.gt_df['frame_id'] == frame_id)
]

boxes = []
labels = []

if len(boxes_data) > 0:
    for _, box_row in boxes_data.iterrows():
        x_min = float(box_row['x_coordinate'])
        y_min = float(box_row['y_coordinate'])
        w = float(box_row['width'])
        h = float(box_row['height'])

        x_max = x_min + w
        y_max = y_min + h

        boxes.append([x_min, y_min, x_max, y_max])
        labels.append(1) # Assuming all objects are class 1. Change if
→ needed.
    else:
        # Handle negative samples (images with no objects)
        boxes = torch.zeros((0, 4), dtype=torch.float32)
        labels = torch.zeros((0,), dtype=torch.int64)

# Convert to Tensors
boxes = torch.as_tensor(boxes, dtype=torch.float32)
labels = torch.as_tensor(labels, dtype=torch.int64)

```

```

image_id = torch.tensor([idx])
area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
iscrowd = torch.zeros((len(labels),), dtype=torch.int64)

target = {}
target["boxes"] = boxes
target["labels"] = labels
target["image_id"] = image_id
target["area"] = area
target["iscrowd"] = iscrowd

# Convert image to tensor (C, H, W)
image = torch.as_tensor(image.transpose((2, 0, 1)), dtype=torch.float32)

return image, target

def __len__(self):
    return len(self.image_data)

# --- 2. Model Definition ---
def get_model(num_classes):
    # Load a model pre-trained on COCO
    model = torchvision.models.detection.
    ↪fasterrcnn_resnet50_fpn(weights="DEFAULT")

    # Replace the classifier with a new one, that has user-defined num_classes
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

    return model

# --- 3. Training Loop ---
def train_one_epoch(model, optimizer, data_loader, device, epoch):
    model.train()
    total_loss = 0

    for i, (images, targets) in enumerate(data_loader):
        images = list(image.to(device) for image in images)
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        # Forward pass (returns losses in train mode)
        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())

        # Backward pass
        optimizer.zero_grad()

```

```

        losses.backward()
        optimizer.step()

        total_loss += losses.item()

        if i % 10 == 0:
            print(f"Epoch: {epoch}, Iter: {i}, Loss: {losses.item():.4f}")

            print(f"Epoch {epoch} finished. Average Loss: {total_loss /
↪len(data_loader):.4f}")

# --- 4. Main Execution ---
def collate_fn(batch):
    return tuple(zip(*batch))

```

```

[16]: DATA_DIR = './dataset'
TRAIN_CSV = './dataset/train.csv'
GT_CSV = './dataset/train_gt.csv'

train_transform = DataAugmentation(is_train=True)
print(f"Initializing Dataset on {DEVICE}...")
dataset = CustomMOTDataset(DATA_DIR, TRAIN_CSV, GT_CSV,
↪transforms=train_transform)

# DataLoader
data_loader = DataLoader(
    dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=4,
    collate_fn=collate_fn
)

print("Initializing Model...")
model = get_model(NUM_CLASSES)
model.to(DEVICE)

# Optimizer
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=LEARNING_RATE, momentum=0.9,
↪weight_decay=0.0005)

# Learning Rate Scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.
↪1)

print("Starting Training...")

```

```

for epoch in range(NUM_EPOCHS):
    train_one_epoch(model, optimizer, data_loader, DEVICE, epoch)
    lr_scheduler.step()

    torch.save(model.state_dict(), f"faster_rcnn_epoch_{epoch}.pth")

print("Training Complete. Model saved.")

```

Initializing Dataset on cuda...

Initializing Model...

Starting Training...

Epoch: 0, Iter: 0, Loss: 1.5182
Epoch: 0, Iter: 10, Loss: 0.6692
Epoch: 0, Iter: 20, Loss: 0.5790
Epoch: 0, Iter: 30, Loss: 0.4700
Epoch: 0, Iter: 40, Loss: 0.4512
Epoch: 0, Iter: 50, Loss: 0.5363
Epoch: 0, Iter: 60, Loss: 0.6896
Epoch: 0, Iter: 70, Loss: 0.5144
Epoch: 0, Iter: 80, Loss: 0.6506
Epoch: 0, Iter: 90, Loss: 0.5967
Epoch: 0, Iter: 100, Loss: 0.4672
Epoch: 0, Iter: 110, Loss: 0.7065
Epoch: 0, Iter: 120, Loss: 0.6198
Epoch: 0, Iter: 130, Loss: 0.5846
Epoch: 0, Iter: 140, Loss: 0.5464
Epoch: 0, Iter: 150, Loss: 0.6010
Epoch: 0, Iter: 160, Loss: 0.5743
Epoch: 0, Iter: 170, Loss: 0.2899
Epoch: 0, Iter: 180, Loss: 0.3799
Epoch: 0, Iter: 190, Loss: 0.4902
Epoch: 0, Iter: 200, Loss: 0.4153
Epoch: 0, Iter: 210, Loss: 0.5327
Epoch: 0, Iter: 220, Loss: 0.4999
Epoch: 0, Iter: 230, Loss: 0.3432
Epoch: 0, Iter: 240, Loss: 0.5042
Epoch: 0, Iter: 250, Loss: 0.5578
Epoch: 0, Iter: 260, Loss: 0.4763
Epoch: 0, Iter: 270, Loss: 0.3972
Epoch: 0, Iter: 280, Loss: 0.4912
Epoch: 0, Iter: 290, Loss: 0.5073
Epoch: 0, Iter: 300, Loss: 0.4757
Epoch: 0, Iter: 310, Loss: 0.3518
Epoch: 0, Iter: 320, Loss: 0.3293
Epoch: 0, Iter: 330, Loss: 0.5363
Epoch: 0, Iter: 340, Loss: 0.5408
Epoch: 0, Iter: 350, Loss: 0.3046
Epoch: 0, Iter: 360, Loss: 0.5140

Epoch: 0, Iter: 370, Loss: 0.4477
Epoch: 0, Iter: 380, Loss: 0.3306
Epoch: 0, Iter: 390, Loss: 0.4233
Epoch: 0, Iter: 400, Loss: 0.3951
Epoch: 0, Iter: 410, Loss: 0.4940
Epoch: 0, Iter: 420, Loss: 0.3952
Epoch: 0, Iter: 430, Loss: 0.3605
Epoch: 0, Iter: 440, Loss: 0.3903
Epoch: 0, Iter: 450, Loss: 0.4589
Epoch: 0, Iter: 460, Loss: 0.3592
Epoch: 0, Iter: 470, Loss: 0.2835
Epoch: 0, Iter: 480, Loss: 0.5315
Epoch: 0, Iter: 490, Loss: 0.4378
Epoch: 0, Iter: 500, Loss: 0.4127
Epoch: 0, Iter: 510, Loss: 0.3907
Epoch: 0, Iter: 520, Loss: 0.3854
Epoch: 0, Iter: 530, Loss: 0.5246
Epoch: 0, Iter: 540, Loss: 0.3563
Epoch: 0, Iter: 550, Loss: 0.2976
Epoch: 0, Iter: 560, Loss: 0.4887
Epoch: 0, Iter: 570, Loss: 0.2880
Epoch: 0, Iter: 580, Loss: 0.5562
Epoch: 0, Iter: 590, Loss: 0.5476
Epoch: 0, Iter: 600, Loss: 0.4419
Epoch: 0, Iter: 610, Loss: 0.4050
Epoch: 0, Iter: 620, Loss: 0.3607
Epoch: 0, Iter: 630, Loss: 0.4379
Epoch: 0, Iter: 640, Loss: 0.3608
Epoch: 0, Iter: 650, Loss: 0.2644
Epoch: 0, Iter: 660, Loss: 0.3911
Epoch: 0, Iter: 670, Loss: 0.4422
Epoch: 0, Iter: 680, Loss: 0.4675
Epoch: 0, Iter: 690, Loss: 0.4621
Epoch: 0, Iter: 700, Loss: 0.4036
Epoch: 0, Iter: 710, Loss: 0.6184
Epoch: 0, Iter: 720, Loss: 0.4278
Epoch: 0, Iter: 730, Loss: 0.4597
Epoch: 0, Iter: 740, Loss: 0.3273
Epoch: 0, Iter: 750, Loss: 0.3009
Epoch: 0, Iter: 760, Loss: 0.3309
Epoch: 0, Iter: 770, Loss: 0.3416
Epoch: 0, Iter: 780, Loss: 0.5150
Epoch: 0, Iter: 790, Loss: 0.3538
Epoch: 0, Iter: 800, Loss: 0.4225
Epoch: 0, Iter: 810, Loss: 0.3406
Epoch: 0, Iter: 820, Loss: 0.2991
Epoch: 0, Iter: 830, Loss: 0.3728
Epoch: 0, Iter: 840, Loss: 0.4728

Epoch: 0, Iter: 850, Loss: 0.4354
Epoch: 0, Iter: 860, Loss: 0.3285
Epoch: 0, Iter: 870, Loss: 0.3399
Epoch: 0, Iter: 880, Loss: 0.3950
Epoch: 0, Iter: 890, Loss: 0.3633
Epoch: 0, Iter: 900, Loss: 0.4393
Epoch: 0, Iter: 910, Loss: 0.3731
Epoch: 0, Iter: 920, Loss: 0.2458
Epoch: 0, Iter: 930, Loss: 0.4006
Epoch: 0, Iter: 940, Loss: 0.2971
Epoch: 0, Iter: 950, Loss: 0.3931
Epoch: 0, Iter: 960, Loss: 0.4401
Epoch: 0, Iter: 970, Loss: 0.4324
Epoch: 0, Iter: 980, Loss: 0.3263
Epoch: 0, Iter: 990, Loss: 0.5149
Epoch: 0, Iter: 1000, Loss: 0.2806
Epoch: 0, Iter: 1010, Loss: 0.2891
Epoch: 0, Iter: 1020, Loss: 0.4556
Epoch: 0, Iter: 1030, Loss: 0.3525
Epoch: 0, Iter: 1040, Loss: 0.3570
Epoch: 0, Iter: 1050, Loss: 0.6598
Epoch: 0, Iter: 1060, Loss: 0.3947
Epoch: 0, Iter: 1070, Loss: 0.3204
Epoch: 0, Iter: 1080, Loss: 0.3702
Epoch: 0, Iter: 1090, Loss: 0.5176
Epoch: 0, Iter: 1100, Loss: 0.4286
Epoch: 0, Iter: 1110, Loss: 0.4323
Epoch: 0, Iter: 1120, Loss: 0.4311
Epoch: 0, Iter: 1130, Loss: 0.2909
Epoch: 0, Iter: 1140, Loss: 0.3532
Epoch: 0, Iter: 1150, Loss: 0.3004
Epoch: 0, Iter: 1160, Loss: 0.3451
Epoch: 0, Iter: 1170, Loss: 0.3863
Epoch: 0, Iter: 1180, Loss: 0.3603
Epoch: 0, Iter: 1190, Loss: 0.3442
Epoch: 0, Iter: 1200, Loss: 0.3595
Epoch: 0, Iter: 1210, Loss: 0.3536
Epoch: 0, Iter: 1220, Loss: 0.3681
Epoch: 0, Iter: 1230, Loss: 0.4468
Epoch: 0, Iter: 1240, Loss: 0.3960
Epoch: 0, Iter: 1250, Loss: 0.5426
Epoch: 0, Iter: 1260, Loss: 0.3496
Epoch: 0, Iter: 1270, Loss: 0.4406
Epoch: 0, Iter: 1280, Loss: 0.3538
Epoch: 0, Iter: 1290, Loss: 0.3545
Epoch: 0, Iter: 1300, Loss: 0.3769
Epoch: 0, Iter: 1310, Loss: 0.5463
Epoch: 0, Iter: 1320, Loss: 0.3024

Epoch: 0, Iter: 1330, Loss: 0.3166
Epoch: 0, Iter: 1340, Loss: 0.2381
Epoch: 0, Iter: 1350, Loss: 0.3796
Epoch: 0, Iter: 1360, Loss: 0.2526
Epoch: 0, Iter: 1370, Loss: 0.2665
Epoch 0 finished. Average Loss: 0.4272
Training Complete. Model saved.

```
[1]: def compute_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])

    union = area1 + area2 - inter
    return inter / union if union > 0 else 0

def smooth_score(prev_score, curr_score, alpha=0.7):
    return alpha * curr_score + (1 - alpha) * prev_score

def center_distance(box1, box2):
    cx1 = (box1[0] + box1[2]) / 2
    cy1 = (box1[1] + box1[3]) / 2
    cx2 = (box2[0] + box2[2]) / 2
    cy2 = (box2[1] + box2[3]) / 2
    return ((cx1 - cx2)**2 + (cy1 - cy2)**2) ** 0.5

def temporal_consistency(prev_dets, curr_dets,
                          iou_thresh=0.3,
                          max_move=50):
    consistent = []

    for curr in curr_dets:
        best_match = None
        best_iou = 0

        for prev in prev_dets:
            iou = compute_iou(prev["box"], curr["box"])
            if iou > best_iou:
                best_iou = iou
                best_match = prev

        if best_iou > iou_thresh:
```

```

        move = center_distance(best_match["box"], curr["box"])
        if move < max_move:
            curr["score"] = smooth_score(
                best_match["score"],
                curr["score"]
            )
            consistent.append(curr)

    return consistent

def format_detections(output):
    dets = []
    for box, score, label in zip(output['boxes'],
                                  output['scores'],
                                  output['labels']):
        dets.append({
            'box': box.tolist(),
            'score': score.item(),
            'label': label.item()
        })
    return dets

```

```
[2]: import torch
```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

```

Using device: cuda

```

[14]: DATA_DIR = './dataset'
TEST_CSV = './dataset/test.csv'
GT_CSV = './dataset/test_det.csv'

print(f"Initializing Dataset on {device}...")
dataset = CustomMOTDataset(DATA_DIR, TEST_CSV, GT_CSV)

# DataLoader
data_loader = DataLoader(
    dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=4,
    collate_fn=collate_fn
)

```

Initializing Dataset on cuda...

```
[15]: # Load the model from saved checkpoint
model = get_model(NUM_CLASSES)
model.load_state_dict(torch.load('faster_rcnn_epoch_0.pth'))
model.to(device)
print("Model loaded successfully from faster_rcnn_epoch_0.pth")
```

Model loaded successfully from faster_rcnn_epoch_0.pth

```
[16]: model.eval()
prev_dets = []

with torch.no_grad():
    for images, _ in data_loader: # labels unused for inference
        images = list(img.to(device) for img in images)
        outputs = model(images)

        curr_dets = format_detections(outputs[0])

        if len(prev_dets) > 0:
            curr_dets = temporal_consistency(prev_dets, curr_dets)

        prev_dets = curr_dets
```

```
[17]: prev_dets
```

```
[17]: [{'box': [411.77532958984375,
185.853515625,
444.6141357421875,
279.27081298828125],
'score': 0.71516140922904,
'label': 1},
{'box': [411.68304443359375,
151.61526489257812,
466.3918151855469,
275.5939025878906],
'score': 0.8471664428710938,
'label': 1},
{'box': [427.0887756347656,
176.78529357910156,
458.6030578613281,
265.2161865234375],
'score': 0.6543741047382355,
'label': 1},
{'box': [495.6076354980469,
189.8789520263672,
522.074951171875,
263.4101867675781],
'score': 0.5265656590461731,
```

```

    'label': 1},
{'box': [430.3477783203125,
        156.52059936523438,
        482.2257995605469,
        271.0824890136719],
 'score': 0.511255842447281,
 'label': 1},
{'box': [487.982666015625,
        191.9164581298828,
        511.01373291015625,
        262.98602294921875],
 'score': 0.45994616448879244,
 'label': 1},
{'box': [107.52800750732422,
        208.10537719726562,
        146.84666442871094,
        275.0344543457031],
 'score': 0.16151764765381812,
 'label': 1},
{'box': [437.28173828125,
        172.36038208007812,
        468.3347473144531,
        266.9402160644531],
 'score': 0.43951584100723273,
 'label': 1},
{'box': [456.1583557128906,
        170.98040771484375,
        518.3236694335938,
        282.9212646484375],
 'score': 0.40068268477916724,
 'label': 1},
{'box': [485.7430419921875,
        156.0769805908203,
        532.62939453125,
        275.83160400390625],
 'score': 0.3321043647825719,
 'label': 1}]

```

[]: